

FLOWPILOT ENGINE

Project Design & Architectural Specification

An AI-powered multi-tenant software project planning and task orchestration platform.

Document Version: 1.1.0 (Task Board & Deletion Sync Updates)

Date: July 16, 2026

Author: Flowpilot Core Team & AI Coding Subagent

Target Environment: macOS / Docker Containerized Dev Environment

1. System Architecture Overview

Flowpilot is organized as a decoupled modern web application consisting of a single-page React frontend running Vite, a FastAPI REST server, and a dual-database model integrating MongoDB for transactional state and LanceDB for vector workspace knowledge.

The transactional MongoDB database maps standard objects (users, projects, milestones, releases, tasks, and retro entries) which are mirrored asynchronously into a LanceDB vector database. When tasks are updated, deleted, or generated, changes trigger back-end events that align the indices automatically.

1.1. Data Models and Schemas

Database schemas are built with Pydantic and beanie. Document relationships are scoped inside the **Project** context:

- **User Document:** Manages credentials and roles (ADMIN, MANAGER, LEAD_DEVELOPER, DEVELOPER, CLIENT). Scoped using standard bcrypt password hashing.
- **Project Document:** The core tenant bucket containing lists of embedded sub-models: Sprints (goals, capacity, duration), Milestones (targets), Releases (deployments), Decisions (Architecture Decision Records), and Retro Entries (went well, improvements, actions).
- **Task Document:** The ticket entity containing title, priority (LOW, MEDIUM, HIGH, CRITICAL), type (EPIC, TASK, SUBTASK, BUG), assignee_id, estimated_hours, sprint_id, and subtask checklist items.
- **Comment Document:** Threaded logs referenced to tasks.

1.2. Role-Based Access Control (RBAC)

The platform enforces strict role validation at the router layer to prevent privilege escalations:

Role	Project Config	Create Task	Update Task Status	Delete Task/Bulk
Client	Read Only	Allowed	Blocked	Blocked
Developer	Read Only	Allowed (Self Assign)	Allowed	Blocked
Lead Dev	Allowed	Allowed (Assign Any)	Allowed	Blocked
Manager	Allowed	Allowed (Assign Any)	Allowed	Allowed
Admin	Allowed	Allowed (Assign Any)	Allowed	Allowed

2. Kanban Task Board & Bulk Operations

The Kanban task board is the primary interface for managing project workflows. It divides issues into columns mapping directly to task states: **TODO**, **IN_PROGRESS**, **IN_REVIEW**, and **DONE**.

To support rapid development cycles, the platform implements unified bulk transitions and deletions, avoiding separate HTTP request overhead for each task card manipulation.

2.1. Backend API Implementations

The FastAPI backend supports transactional bulk status updates and bulk task deletions via the following REST schemas:

```
# Bulk status update endpoint
@router.post("/bulk-status")
async def bulk_update_task_status(payload: BulkStatusUpdateSchema, ...):
    # Triggers status mutations, updates updated_at, registers ActivityLogs,
    # and dispatches sync events to LanceDB asynchronously in the background.

# Bulk delete endpoint (Restricted to Manager/Admin)
@router.post("/bulk-delete")
async def bulk_delete_tasks(payload: BulkDeleteSchema, ...):
    # Performs transactional deletions, triggers background cleanup tasks,
    # and emits sync event 'delete' to mirror vector space changes.
```

2.2. Frontend UI and Multi-Selection Flow

The React Kanban board features a dual-layer select-all mechanism with custom overlays:

- **Board-Level Select All:** A checkbox situated in the main filter bar. Toggling it checks/unchecks all tasks currently matching the search filters.
- **Column-Level Select All:** Checkboxes in the header of each status column (e.g. Backlog, In Progress). Toggling check/uncheck selects only tasks inside that specific status column.
- **Floating Action Panel:** A glassmorphic sticky panel slide-in appears at the bottom center of the viewport once one or more checkboxes are checked. It exposes batch actions: To Do, In Prog, Review, Done, Delete, and Cancel. Deletions trigger a custom modal window confirming the actions in detail rather than browser defaults.

3. AI Copilot RAG & WBS Stream Engine

The Flowpilot application integrates a closed-loop Retrieval-Augmented Generation (RAG) system with a deterministic routing loop to query system context, documents, and task boards directly.

3.1. Unified Knowledge Index and Retriever

The platform operates a **HybridRetriever** blending keyword and vector search over a unified knowledge table inside LanceDB. When query context is loaded, elements undergo the following pipeline:

1. **Vector Search:** Embeds the user query via ModernBERT (768 dimensions) and queries LanceDB matching to an inclusive cosine similarity threshold (default 0.45). For backlog/task count searches, a lower threshold of 0.35 is applied to capture general intent.
2. **Keyword Search:** Tokenizes the query text and scans the local LanceDB index for overlap score, filtered strictly by target project_id and entity_type.
3. **Reciprocal Rank Fusion (RRF):** Merges rank indexes of both results to surface the most relevant elements.

3.2. Autorun MongoDB Authoritative Fallback

Because semantic search can fail on numerical count questions (e.g. *'how many tasks are left to do?'*), the backlog search tool is designed with a MongoDB authoritative fetch. The engine always queries MongoDB for the authoritative status lists, then blends it with vector search rankings. This guarantees 100% accurate status counts even if vector database indices are stale or rebuilding.

3.3. Work Breakdown Structure (WBS) Generation

The WBS engine reads unstructured text requirements, feeds the context to the local LLM running on Apple Silicon, and outputs structural project task hierarchies in a JSON format streamed via Server-Sent Events (SSE). The generated WBS can be committed directly to create the project's task board backlog in one click.